



ΚΡΥΠΤΟΓΡΑΦΙΑ

Γ' Εξάμηνο – Εργασία 1



NOVEMBER 15, 2024

Μαρίνος Ματθαίου – inf2023004

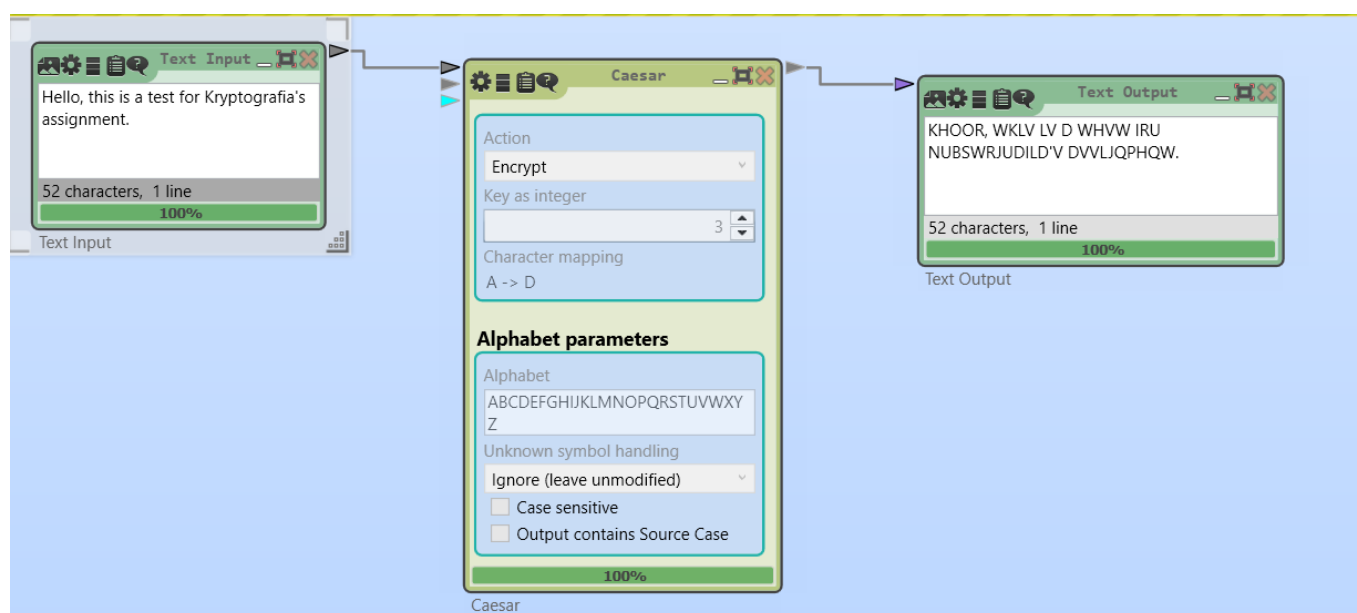
Περιγραφή Εργασίας

Στην 1^η εργασία που μας ανατέθηκε στην Κρυπτογραφία, κληθήκαμε να υλοποιήσουμε κάποιους αλγόριθμους μέσω του Cryptool. Συγκεκριμένα, χωρισμένα σε ενότητες, χρειάστηκε να αναλύσουμε τους αλγόριθμους Caesar, Affine και Vigenere περιγράφοντας αναλυτικά τη διαδικασία. Στη συνέχεια, απαραίτητο ήταν να προσθέσουμε την δοκιμή συχνότητας (Frequency Test) στον καθένα από τους αλγόριθμους και να συγκρίνουμε τα αποτελέσματα καθώς και τις διαφορές που παρατηρήσαμε μεταξύ τους. Μετέπειτα, τέθηκε αναγκαίο να υλοποιήσουμε επίθεση ωμής βίας στους αλγόριθμους Caesar και Vigenere επεξηγώντας αναλυτικά τα βήματα που ακολουθήσαμε. Τελειώνοντας, φτιάξαμε και χρησιμοποιήσαμε κώδικα στην Python, χωρίς τις επιθέσεις, για τον καθένα αλγόριθμο ξεχωριστά και τους πραγματοποιήσαμε με ένα frequency test.

Ενότητα Α) Ανάλυση και Περιγραφή των Αλγόριθμων Κρυπτογράφησης

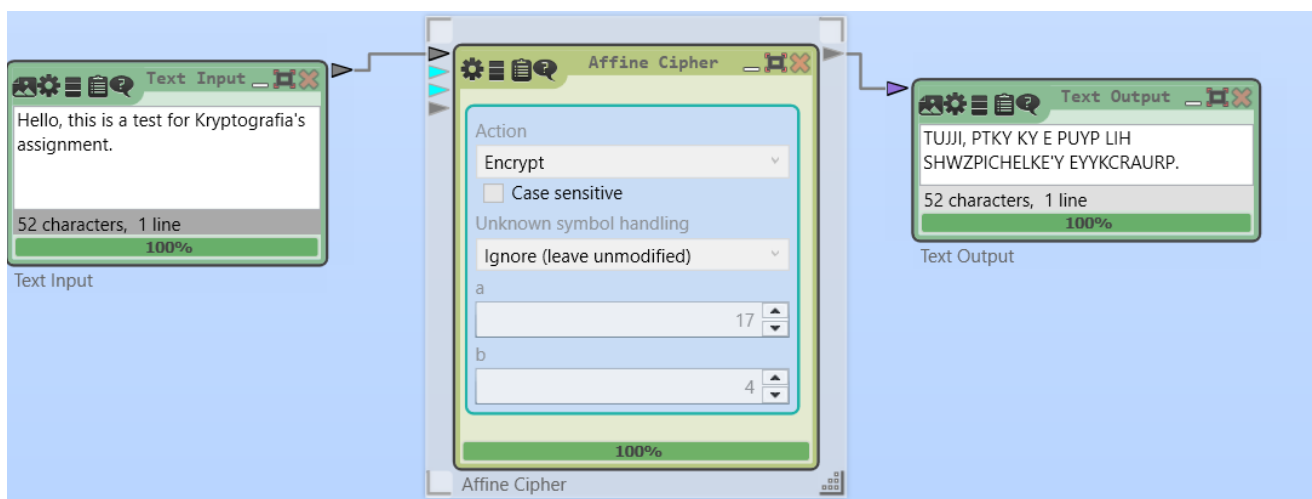
Αρχικά, ξεκινήσαμε με την θεωρητική ανάλυση των αλγορίθμων Caesar, Affine και Vigenere μέσω του Cryptool καθώς και τον τρόπο που ο καθένας από αυτούς εφαρμόζουν την κρυπτογράφηση μεταθέτοντας γράμματα.

- (1) Αλγόριθμος Caesar: Λειτουργεί με μια απλή μετατόπιση γραμμάτων κατά έναν συγκεκριμένο αριθμό θέσεων, ο οποίος αποτελεί και το κλειδί.



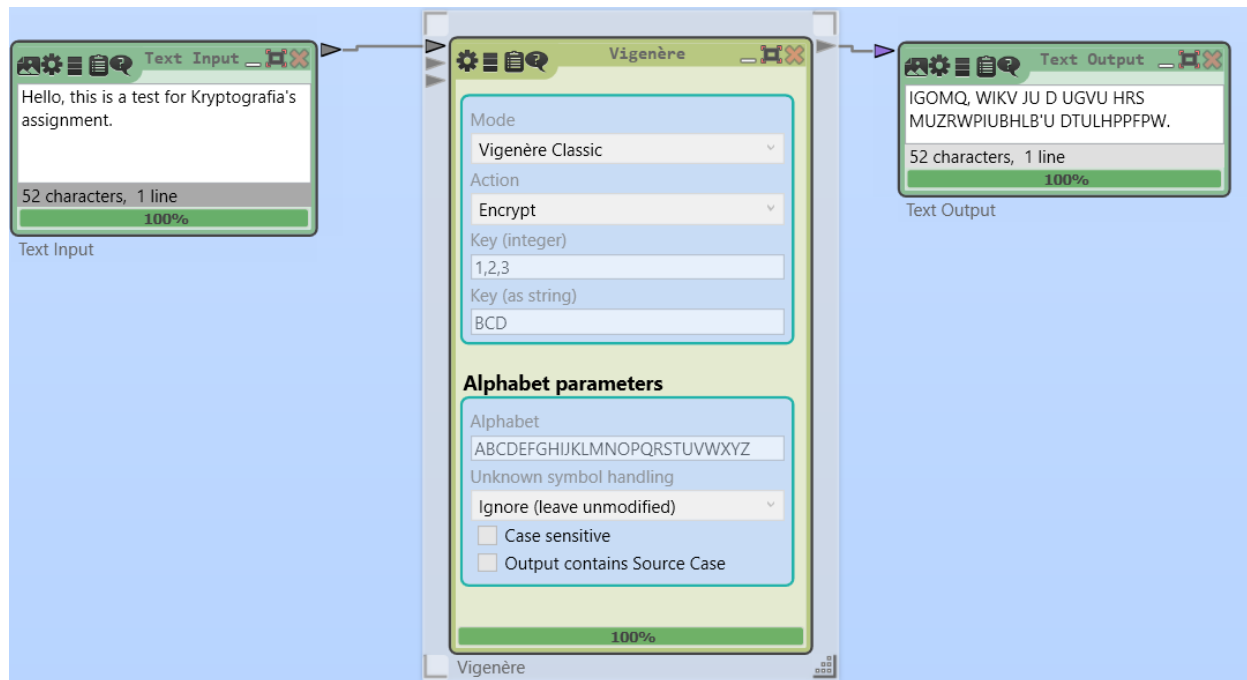
Στην συγκεκριμένη περίπτωση, χρησιμοποιήθηκε ένα Text Input που είχε ως κείμενο «Hello, this is a test for Kryptografia's assignment.», που ενώθηκε με την διαθέσιμη ταμπλέτα του αλγόριθμου Caesar. Στον αλγόριθμο χρησιμοποιήθηκε το Encrypt, που δηλώνει Κρυπτογραφία στα ελληνικά, και ως κλειδί υποδηλώσαμε το 3. Στην συνέχεια, προσθήσαμε ένα Text Output που συνδέθηκε με τον Caesar έτσι ώστε να πραγματοποιηθεί η κρυπτογράφηση και να μας δείξει το αποτέλεσμα. Αφού συνδέθηκαν όλα με επιτυχία μεταξύ τους, το τρέξαμε και είδαμε τα ανάλογα αποτελέσματα.

(2) Αλγόριθμος Affine: Η κρυπτογράφηση γίνεται με μια γραμμική συνάρτηση, η οποία είναι $E(x) = (a * x + b) \bmod 26$, όπου το a και το b είναι κλειδιά που πληρούν ορισμένες μαθηματικές συνθήκες.



Στην συγκεκριμένη περίπτωση, όπως και στον αλγόριθμο Caesar, χρησιμοποιήθηκε ένα Text Input που είχε ως κείμενο «Hello, this is a test for Kryptografia's assignment.», που ενώθηκε με την διαθέσιμη ταμπλέτα του αλγόριθμου Affine. Στον αλγόριθμο χρησιμοποιήθηκε το Encrypt και ως κλειδί 'a' το 17 ενώ ως κλειδί 'b' το 4. Το 'a' είναι πολλαπλασιαστικός παράγοντας και το 'b' είναι η μετατόπιση. Το 'a' πρέπει να είναι μια τιμή που είναι αντιστρέψιμη σε σχέση με το αλφάβητο (π.χ. για το λατινικό αλφάβητο, αποδεκτές τιμές είναι αυτές που είναι πρώτες με το 26, όπως 1, 3, 5, 7, 9, κ.λπ.). Αφού επεξεργαστήκαμε τον λογάριθμο με τα δεδομένα μας, προσθήσαμε σε αυτόν ένα Text Output όπου μας εμφάνισε το κρυπτογραφημένο κείμενο.

(3) Αλγόριθμος Vigenere: Βασίζεται σε έναν πίνακα πολυ-αλφαβητικών υποκαταστάσεων, χρησιμοποιώντας μια λέξη-κλειδί για την κρυπτογράφηση με μετατοπίσεις που διαφέρουν για κάθε γράμμα του κειμένου. Οι αλγόριθμοι αυτοί διαφέρουν στη μεθοδολογία κρυπτογράφησης και είναι χρήσιμοι για διαφορετικές εφαρμογές λόγω της απλότητας ή πολυπλοκότητάς τους.

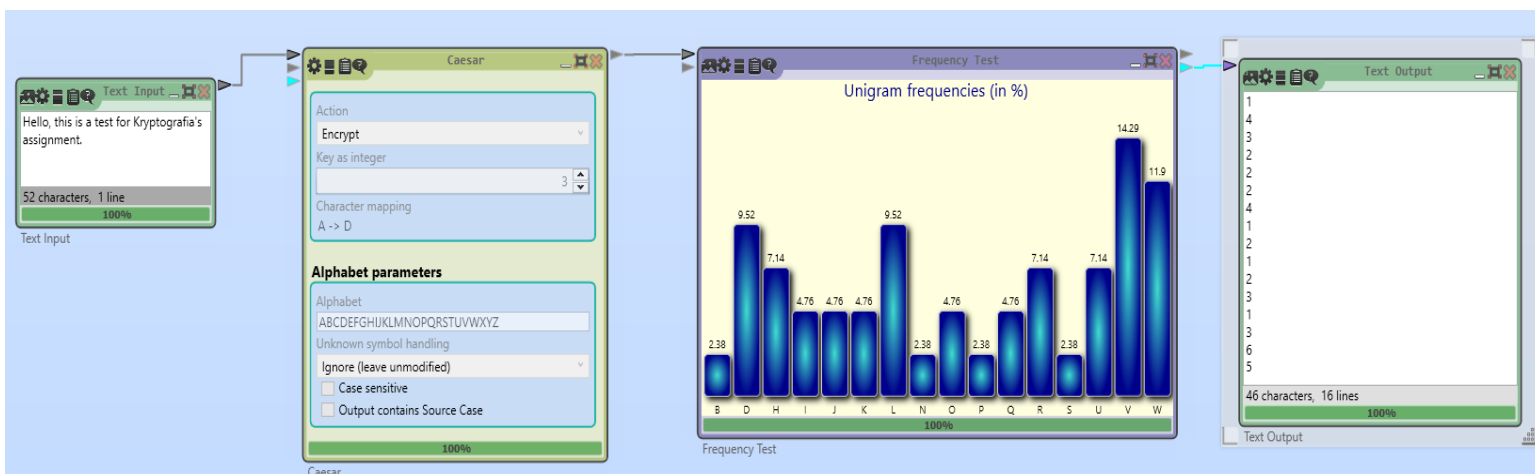


Στην συγκεκριμένη περίπτωση, όπως και στους προηγούμενους αλγόριθμους χρησιμοποιήθηκε ένα Text Input που είχε ως κείμενο «Hello, this is a test for Kryptografia's assignment.», που ενώθηκε με την διαθέσιμη ταμπλέτα του αλγόριθμου Vigenere. Στον αλγόριθμο χρησιμοποιήθηκε το Encrypt, με κλειδί αριθμού 1,2,3 και κλειδί λέξης BCD. Ο αλγόριθμος βασίζεται σε ένα κλειδί που αποτελείται από λέξεις και χρησιμοποιεί διαφορετική μετατόπιση για κάθε χαρακτήρα του κλειδιού. Αφού επεξεργαστήκαμε τον λογάριθμο με τα δεδομένα μας, προσθέσαμε σε αυτόν ένα Text Output όπου μας εμφάνισε το κρυπτογραφημένο κείμενο.

Ενότητα Β) Χρήση του Frequency Test και σύγκριση αποτελεσμάτων

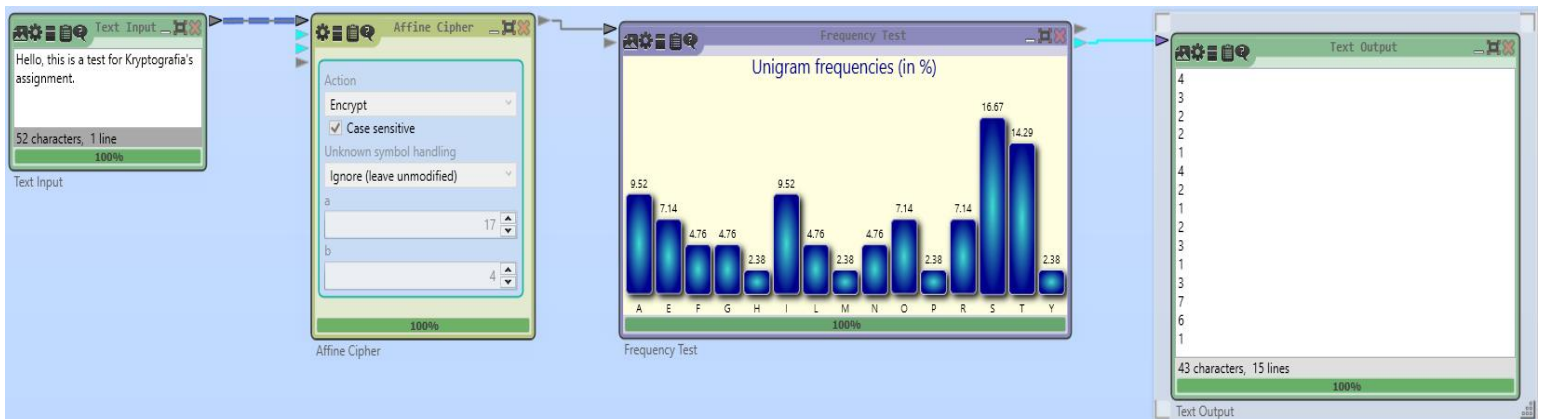
Στη συνέχεια, μετά την ολοκλήρωση των αποκρυπτογραφήσεων προχωρήσαμε στην δοκιμή συχνότητας για να εξετάσουμε τις στατιστικές κατανομές των χαρακτήρων στα κρυπτογραφημένα κείμενα. Το Frequency Test είναι ένα χρήσιμο εργαλείο που αναλύει πόσο εμφανίζεται κάθε γράμμα στο κείμενο και το συγκρίνει με τη φυσική κατανομή της γλώσσας.

- (1) Caesar Cipher: Το εφαρμόσαμε με Text Input όπως αναφέρθηκε παραπάνω, και ένα συγκεκριμένο κλειδί (3). Μετέπειτα εκτελέσαμε το Frequency Test στο κρυπτογραφημένο κείμενο και είδαμε τα αποτελέσματα.



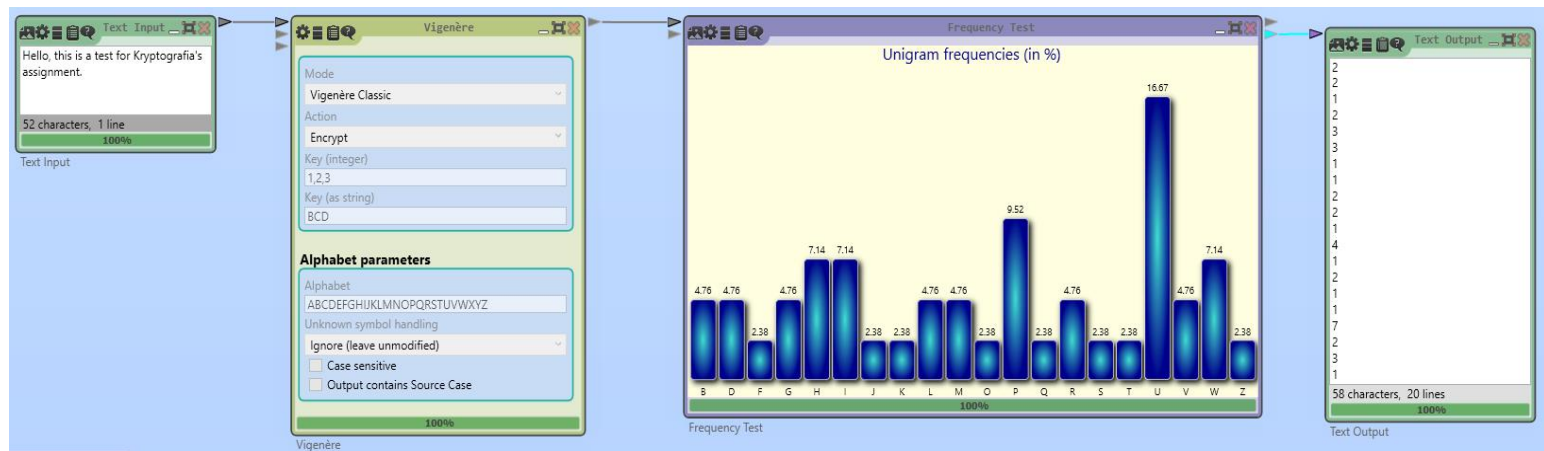
Παρατηρήσαμε ότι η κατανομή των γραμμάτων παρέμεινε ίδια, αλλά μετατοπίστηκε λόγω του αλγορίθμου. Αυτό καθιστά το Caesar Cipher ευάλωτο σε ανάλυση συχνότητας.

(2) Affine Cipher: Δοκιμάσαμε το Frequency Test στα κρυπτογραφημένα κείμενα.



Παρόμοια με τον Caesar, η κατανομή των γραμμάτων επηρεάστηκε, αλλά η συνολική συχνότητα παρέμεινε ίδια. Οι συχνότεροι χαρακτήρες στο αρχικό κείμενο παρέμειναν συχνοί στο κρυπτογραφημένο, απλώς μετατοπίστηκαν. Αυτό επιβεβαίωσε ότι και ο Affine Cipher είναι ευάλωτος στην ίδια μέθοδο ανάλυσης.

(3) Vigenere Cipher: Εκτελέσαμε το Frequency Test στα κρυπτογραφημένα κείμενα.



Εδώ παρατηρήσαμε σημαντική διαφορά: η κατανομή των γραμμάτων ήταν πιο ομοιόμορφη. Αυτό συμβαίνει επειδή το Vigenere χρησιμοποιεί διαφορετικές μετατοπίσεις για κάθε γράμμα (ανάλογα με τη λέξη-κλειδί), καθιστώντας πιο δύσκολο να αναγνωριστούν τα στατιστικά μοτίβα.

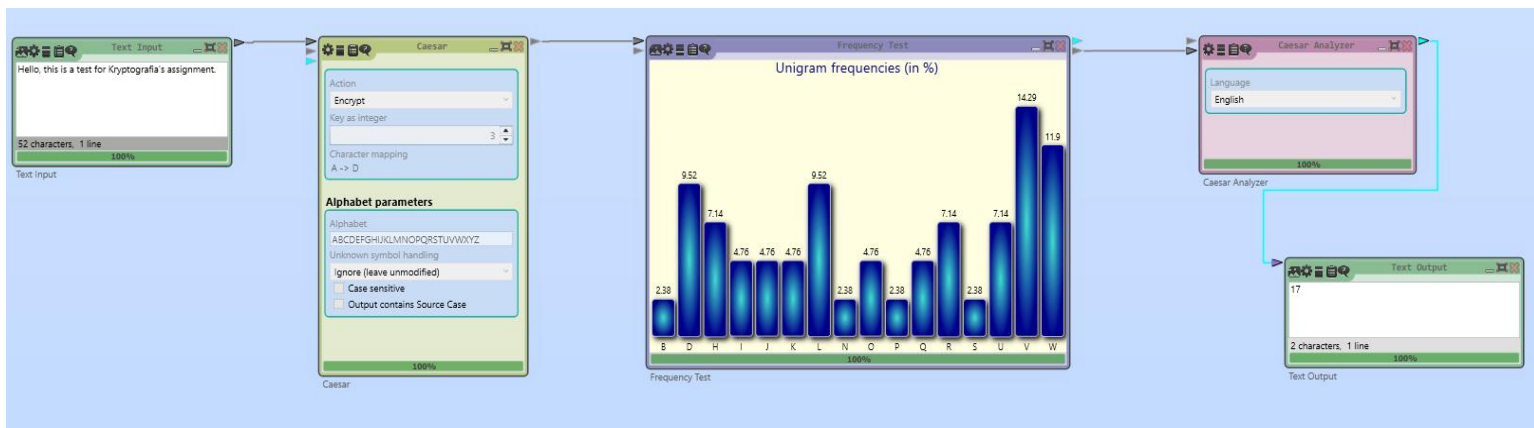
Διαφορές Αλγορίθμων:

- Caesar και Affine: Τα κρυπτογραφημένα κείμενα διατηρούν τα στατιστικά χαρακτηριστικά του αρχικού κειμένου, καθιστώντας τα ευάλωτα σε ανάλυση συχνότητας.
- Vigenere: Το αποτέλεσμα εξαρτάται από το μήκος της λέξης-κλειδιού. Με μικρές λέξεις-κλειδιά, παραμένουν κάποια μοτίβα. Με μεγαλύτερες λέξεις-κλειδιά, η στατιστική κατανομή γίνεται πιο επίπεδη, καθιστώντας δυσκολότερη την ανάλυση.

Ενότητα Γ: Υλοποίηση Επίθεση Ωμής Βίας και Ανάλυση Συχνότητας

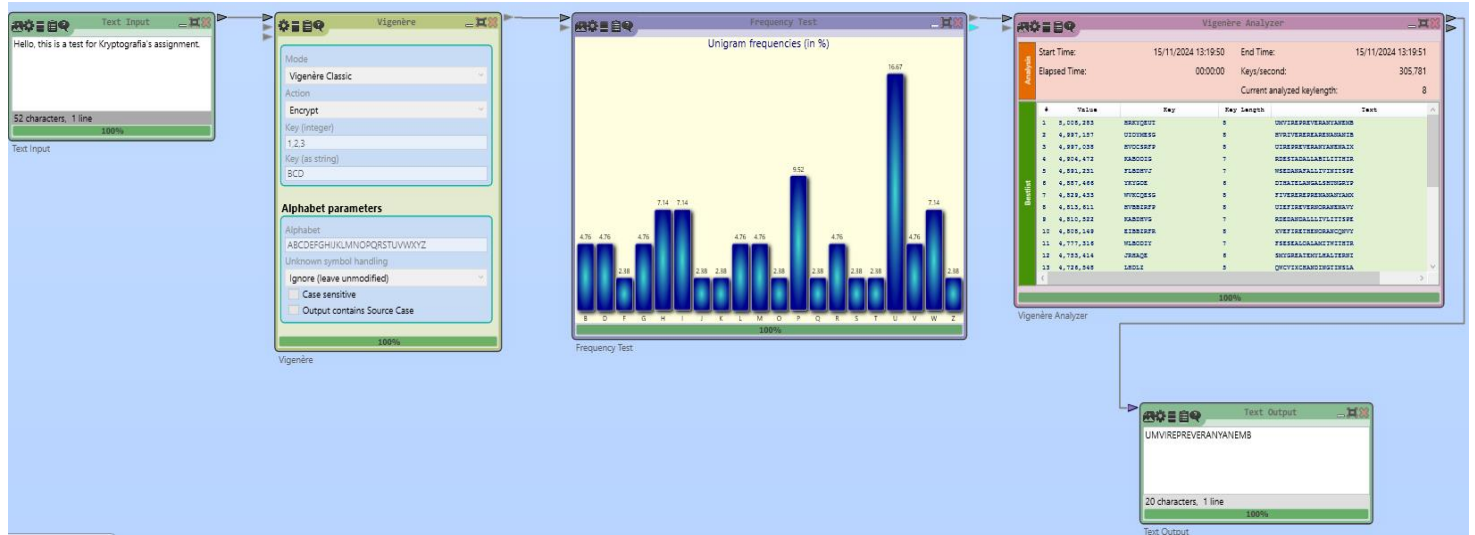
Υλοποιήσαμε επίθεση ωμής βίας και ανάλυση συχνότητας για κάθε αλγόριθμο, χρησιμοποιώντας τα components Caesar Analysis και Vigenere Analysis. Μετά από οδηγία του διδάσκοντα και έλλειψη component για Affine Analysis, δεν αναφερθήκαμε στον συγκεκριμένο αλγόριθμο.

(1) Caesar Analyzer: Εκτελέσαμε την κρυπτογράφηση και λάβαμε το κρυπτογραφημένο κείμενο.



Από την Ενότητα (B), προσθήσαμε το Caesar Analyzer ανάμεσα από το Frequency Test και το Text Output. Δηλαδή, να ενώνεται με το Frequency Test και να εξάγει το αποτέλεσμα, αφού υλοποιηθεί πρώτα το Analyzer, στο Text Output. Εξετάσαμε πώς η μετατόπιση άλλαξε τα γράμματα του κειμένου και επιβεβαιώσαμε τη λειτουργικότητα του αλγορίθμου.

(2) Vigenere Analyzer: Επιλέξαμε τη μέθοδο Vigenere και προσθήσαμε μια λέξη-κλειδί (BCD). Κρυπτογραφήσαμε ένα κείμενο για να παρατηρήσουμε πώς το μήκος της λέξης-κλειδιού επηρεάζει το αποτέλεσμα.



Vigenère Analyzer

Mode: Classic Vigenère

Lower bound of keylength: 1

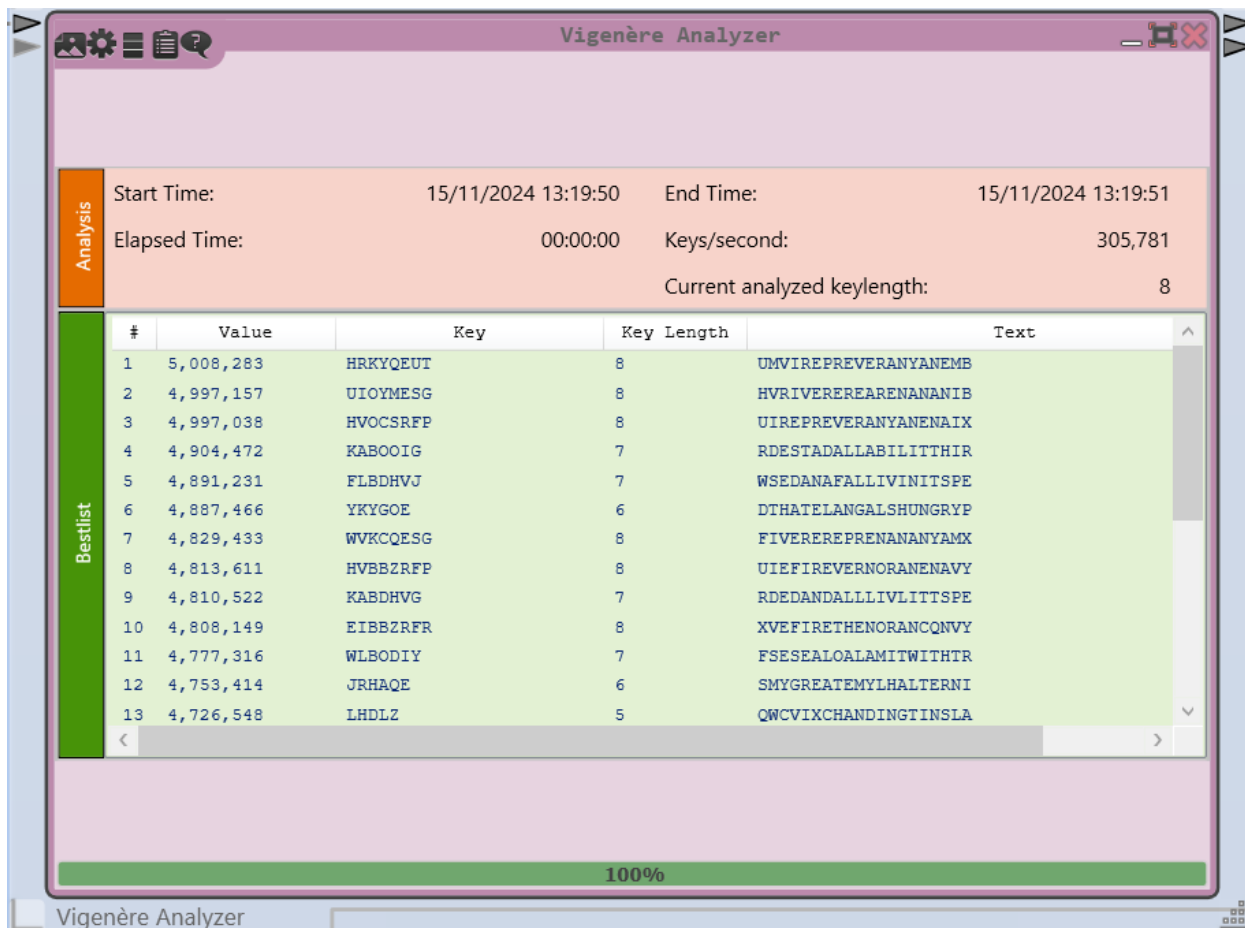
Upper bound of keylength: 8

Restarts: 50

Language: English

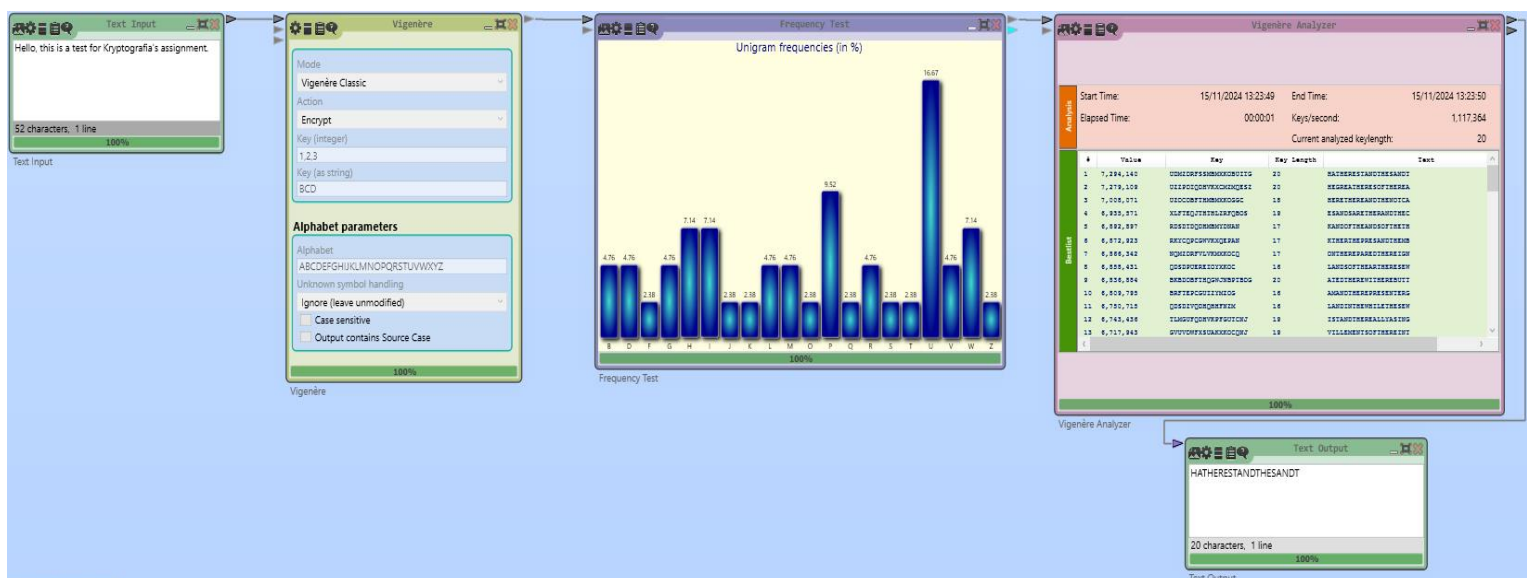
Type of n-grams: Pentagrams

Keystyle: Random



Σε αυτό το παράδειγμα χρησιμοποιήσαμε σαν keylength το 1 μέχρι το 8. Το Vigenere Analyzer μας δείχνει τα πιθανά αποτελέσματα μετά την υλοποίηση τους. Αναφέρει επίσης πόσα κλειδιά δημιουργήθηκαν κατά τον τερματισμό της υλοποίησης, όπως και τον χρόνο που χρειάστηκε.

Πάμε σε ένα άλλο παράδειγμα τώρα, με keylength το 4 μέχρι το 20.



Vigenère Analyzer

Mode

Classic Vigenère

Lower bound of keylength

4

Upper bound of keylength

20

Restarts

50

Language

English

Type of n-grams

Pentagrams

Keystyle

Random

100%

Vigenère Analyzer					
Analysis	Start Time:		15/11/2024 13:23:49	End Time:	15/11/2024 13:23:50
	Elapsed Time:		00:00:01	Keys/second:	1,117,364
	Current analyzed keylength:				20
Bestlist	#	Value	Key	Key Length	Text
	1	7,294,140	UDMZDRFSSMBMXKOBUITG	20	HATHERESTANDTHESANDT
	2	7,279,109	UZZPDIQDHVKXCMZMQESZ	20	HEGREATHERESOFTHAREA
	3	7,008,071	UZOCOBFTHMBMXKOGGC	18	HERETHEREANDTHENOTCA
	4	6,935,571	XLFTQJTHLZRFQBOS	19	ESANDSARETHERANDTHEC
	5	6,892,897	RSDTDQDHMBMYDNAN	17	KANDOFTHEANDSOFTHETH
	6	6,872,923	RKYCQPCGWVKXQEPAN	17	KTHERTHEPRESANDTHEMB
	7	6,866,342	NQMZDRFVLVKMXKOCQ	17	ONTHEREPAREDTHEREIGN
	8	6,858,431	QSDPUEREIOYXKOC	16	LANDSOFTHEARTHERESEW
	9	6,836,884	BKBDQBFTHQGWJNBPTBDG	20	ATEDTHEREWITHEREBUTT
	10	6,809,795	BRFTEPCGUIZYMZOG	16	AMANDTHEREPRESENTERG
	11	6,750,715	QSDSZVQDQHHPFNZM	16	LANDINTHEWHILETHESEW
	12	6,743,436	TLMGUFQDHVKPFGUTCNJ	19	ISTANDTHEREALLYASING
	13	6,717,943	GVUVDWFXSUAKXKOCQNJ	19	VILLEMENTSOFTHEREINT
100%					

Με αυτό το παράδειγμα, δοκιμάσαμε διαφορετικά μήκη λέξεων-κλειδιών και εξηγήσαμε γιατί μεγαλύτερες λέξεις-κλειδιά προσφέρουν μεγαλύτερη ασφάλεια μιας και τα αποτελέσματα είναι περισσότερα, που το καθιστά δυσκολότερο ως προς την ανάλυση του.

Ενότητα Δ: Αλγόριθμοι σε Python

Στο τελευταίο βήμα, υλοποιήσαμε ένα frequency test για την ανάλυση της συχνότητας γραμμάτων σε οποιοδήποτε κείμενο, ανεξάρτητα από τον τύπο του αλγόριθμου κρυπτογράφησης που εφαρμόστηκε. Το frequency test μετρά την εμφάνιση κάθε γράμματος στο κείμενο, υπολογίζοντας το ποσοστό του επί του συνόλου. Αυτό το test εφαρμόζεται στα κρυπτογραφημένα κείμενα από Caesar, Affine ή Vigenere, παρέχοντας ενδείξεις για το ποιο από τα αποτελέσματα είναι πιο πιθανό να αποτελεί σωστή αποκρυπτογράφηση. Αυτή η μέθοδος ανάλυσης είναι ιδιαίτερα χρήσιμη σε περιπτώσεις όπου δεν έχουμε συγκεκριμένο κλειδί και μας βοηθά να συγκρίνουμε πιθανές αποκρυπτογραφήσεις με βάση τη συχνότητα των γραμμάτων.

(1) Caesar Code:

C:\> Users > User > Desktop > Γ' Εξάμηνο > Κρυπτογραφία > Εργασίες > 1 > Codes > Caesar code.py > ...

```
1 def caesar_encrypt(plaintext, shift):
2     ciphertext = ""
3     for char in plaintext:
4         if char.isalpha():
5             shifted = (ord(char.lower()) - ord('a') + shift) % 26 + ord('a')
6             ciphertext += chr(shifted) if char.islower() else chr(shifted).upper()
7         else:
8             ciphertext += char
9     return ciphertext
10
11 def caesar_decrypt(ciphertext, shift):
12     return caesar_encrypt(ciphertext, -shift)
13
14 plaintext = "Hello, this is a test for Kryptografia's assignment."
15 shift = 3
16 encrypted_text = caesar_encrypt(plaintext, shift)
17 decrypted_text = caesar_decrypt(encrypted_text, shift)
18
19 print("\nCaesar Cipher:")
20 print("Plaintext:", plaintext)
21 print("Encrypted:", encrypted_text)
22 print("Decrypted:", decrypted_text)
23
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\User\Desktop\Γ' Εξάμηνο\Κρυπτογραφία\Εργασίες\1\Codes>
& 'c:\Users\User\AppData\Local\Microsoft\WindowsApps\python3.11.exe' 'c:\Users\User\.vscode\ext
..\debugpy\launcher' '56399' '--' 'C:\Users\User\Desktop\Γ' Εξάμηνο\Κρυπτογραφία\Εργασίες\1\
```

```
Caesar Cipher:
Plaintext: Hello, this is a test for Kryptografia's assignment.
Encrypted: Koor, wlv lv d whvw iru Nubswrjudild'v dvljqphqw.
Decrypted: Hello, this is a test for Kryptografia's assignment.
```

Caesar Code with Frequency:

C: > Users > User > Desktop > Γ' Εξάμηνο > Κρυπτογραφία > Εργασίες > 1 > Codes > Frequency test code.py > ...

```
1 from collections import Counter
2
3 def frequency_test(text):
4     frequencies = Counter(text)
5     total_letters = sum(frequencies.values())
6     for letter, count in frequencies.items():
7         print(f"{letter}: {count / total_letters * 100:.2f}%")
8
9 ciphertext = "Khoor, wklv lv d whvw iru Nubswrjudild'v dvvljqphqw." #text to decrypt
10 print("Frequency test on Caesar:")
11 frequency_test(ciphertext)
12
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Frequency test on Caesar:

K: 1.92%
h: 5.77%
o: 3.85%
r: 5.77%
,: 1.92%
: 13.46%
w: 9.62%
k: 1.92%
l: 7.69%
v: 11.54%
d: 7.69%
i: 3.85%
u: 5.77%
N: 1.92%
b: 1.92%
s: 1.92%
j: 3.85%
' : 1.92%
q: 3.85%
p: 1.92%
.: 1.92%

(2) Affine Code:

C: > Users > User > Desktop > Γ' Εξάμηνο > Κρυπτογραφία > Εργασίες > 1 > Codes > Affine code.py > affine_decrypt > ciphertex

```
1 def affine_encrypt(plaintext, a, b):
2     return "".join(
3         chr(((a * (ord(char) - ord('a')) + b) % 26) + ord('a')) if char.islower()
4         else chr(((a * (ord(char) - ord('A')) + b) % 26) + ord('A')) if char.isupper()
5         else char for char in plaintext
6     )
7
8 def affine_decrypt(ciphertext, a, b):
9     m = 26
10    inv_a = pow(a, -1, m) #reverse of a mod 26
11    return "".join(
12        chr(((inv_a * (ord(char) - ord('a')) - b)) % 26) + ord('a')) if char.islower()
13        else chr(((inv_a * (ord(char) - ord('A')) - b)) % 26) + ord('A')) if char.isupper()
14        else char for char in ciphertext
15    )
16
17 plaintext = "Hello, this is a test for Kryptografia's assignment."
18 a, b = 17, 4
19 encrypted_text = affine_encrypt(plaintext, a, b)
20 decrypted_text = affine_decrypt(encrypted_text, a, b)
21
22 print("\nAffine Cipher:")
23 print("Plaintext:", plaintext)
24 print("Encrypted:", encrypted_text)
25 print("Decrypted:", decrypted_text)
26
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
../../debugpy\launcher '56436' '--' 'C:\Users\User\Desktop\Γ' Εξάμηνο\Κρυπτογραφία\Εργασίες\1\Codes\Affine cod
```

Affine Cipher:

Plaintext: Hello, this is a test for Kryptografia's assignment.

Encrypted: Tujji, ptky ky e puyp lih Shwzpichelke'y eykcraurp.

Decrypted: Hello, this is a test for Kryptografia's assignment.

Affine Code With Frequency:

C: > Users > User > Desktop > Γ' Εξάμηνο > Κρυπτογραφία > Εργασίες > 1 > Codes > Frequency test code.py > ...

```
1 from collections import Counter
2
3 def frequency_test(text):
4     frequencies = Counter(text)
5     total_letters = sum(frequencies.values())
6     for letter, count in frequencies.items():
7         print(f"{letter}: {count / total_letters * 100:.2f}%")
8
9 ciphertext = "Tujji, ptky ky e puyp lih Shwzpichelke'y eykcraurp." #text to decrypt
10 print("Frequency test on Affine:")
11 frequency_test(ciphertext)
12
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
../../debugpy\launcher '56461' '--' 'C:\Users\User\Desktop\Γ' Εξάμηνο\Κρυπτογραφία\Εργασίες\1\Codes\Frequency test code. |
```

Frequency test on Affine:

T: 1.92%

u: 5.77%

j: 3.85%

i: 5.77%

,: 1.92%

: 13.46%

p: 9.62%

t: 1.92%

k: 7.69%

y: 11.54%

e: 7.69%

l: 3.85%

h: 5.77%

S: 1.92%

w: 1.92%

z: 1.92%

c: 3.85%

': 1.92%

r: 3.85%

a: 1.92%

.: 1.92%

(3) Vigenere Code:

C: > Users > User > Desktop > Γ' Εξάμηνο > Κρυπτογραφία > Εργασίες > 1 > Codes > Vigenere code.py > ...

```
1 def vigenere_encrypt(plaintext, key):
2     key_repeated = (key * (len(plaintext) // len(key) + 1))[:len(plaintext)]
3     return "".join(
4         chr((ord(char) - ord('a') + ord(k) - ord('a')) % 26 + ord('a')) if char.islower()
5         else chr((ord(char) - ord('A') + ord(k.upper()) - ord('A')) % 26 + ord('A')) if char.isupper()
6         else char for char, k in zip(plaintext, key_repeated)
7     )
8
9 def vigenere_decrypt(ciphertext, key):
10    key_repeated = (key * (len(ciphertext) // len(key) + 1))[:len(ciphertext)]
11    return "".join(
12        chr((ord(char) - ord('a') - (ord(k) - ord('a')) + 26) % 26 + ord('a')) if char.islower()
13        else chr((ord(char) - ord('A') - (ord(k.upper()) - ord('A')) + 26) % 26 + ord('A')) if char.isupper()
14        else char for char, k in zip(ciphertext, key_repeated)
15    )
16
17 plaintext = "Hello, this is a test for Kryptografia's assignment."
18 key = "BCD"
19 encrypted_text = vigenere_encrypt(plaintext, key)
20 decrypted_text = vigenere_decrypt(encrypted_text, key)
21
22 print("\nVigenere Cipher:")
23 print("Plaintext:", plaintext)
24 print("Encrypted:", encrypted_text)
25 print("Decrypted:", decrypted_text)
26
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\User\Desktop\Γ' Εξάμηνο\Κρυπτογραφία\Εργασίες\1\Codes>
& 'c:\Users\User\AppData\Local\Microsoft\WindowsApps\python3.11.exe' 'c:\Users\User\.vscode\extensions\ms-python.debugpy-2023.12.0\bin\debugpy_launcher' '56479' '--' 'c:\Users\User\Desktop\Γ' Εξάμηνο\Κρυπτογραφία\Εργασίες\1\Codes\Vigenere code.py'
```

```
Vigenere Cipher:
Plaintext: Hello, this is a test for Kryptografia's assignment.
Encrypted: Iaigk, pedo do v qzoq blm Nmumokdmwcdw'n xnofbjjzjq.
Decrypted: Hello, this is a test for Kryptografia's assignment.
```

Vigenere Code with Frequency:

C: > Users > User > Desktop > Γ' Εξάμηνο > Κρυπτογραφία > Εργασίες > 1 > Codes > Frequency test code.py > ...

```
3 def frequency_test(text):
4     frequencies = Counter(text)
5     total_letters = sum(frequencies.values())
6     for letter, count in frequencies.items():
7         print(f"{letter}: {count / total_letters * 100:.2f}%")
8
9 ciphertext = "Iaigk, pedo do v qzoq blm Nmumokdmwcdw'n xnofbjjzjq." #text to decrypt
10 print("Frequency test on Vigenere:")
11 frequency_test(ciphertext)
12
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
I: 1.92%
a: 1.92%
i: 1.92%
g: 1.92%
k: 3.85%
,: 1.92%
: 13.46%
p: 1.92%
e: 1.92%
d: 7.69%
o: 9.62%
v: 1.92%
q: 5.77%
z: 3.85%
b: 3.85%
l: 1.92%
m: 7.69%
N: 1.92%
u: 1.92%
w: 3.85%
c: 1.92%
': 1.92%
n: 3.85%
x: 1.92%
f: 1.92%
j: 5.77%
.: 1.92%
```

Αποτελέσματα:

Το CrypTool μας βοήθησε να κατανοήσουμε την πρακτική εφαρμογή των αλγορίθμων, παρέχοντας ένα διαδραστικό περιβάλλον για δοκιμές. Μέσα από τις διαδικασίες κρυπτογράφησης και αποκρυπτογράφησης, παρατηρήσαμε τις διαφορές στην ασφάλεια, την πολυπλοκότητα, και τη στατιστική ανάλυση που προσφέρουν οι αλγόριθμοι, ενώ μάθαμε πώς να προσαρμόζουμε παραμέτρους για την επίτευξη καλύτερων αποτελεσμάτων.